



8. Corrutinas

Física e Inteligencia Artificial para Videojuegos

Doble Grado en Física e Inteligencia Artificial

Curso 2025 – 2026

Fernando Madrid Recio

Corrutinas

- A la hora de establecer tareas más complejas en el tiempo, es preferible utilizar **corrutinas**.
- Podemos entender las corrutinas como métodos que se ejecutan por **intervalos de tiempo**.
- Un claro ejemplo:
 - El comportamiento de un semáforo

Corrutinas

```
IEnumerator Semaforo()  
{  
    Debug.Log("Rojo");  
    yield return new WaitForSeconds(3);  
    Debug.Log("Amarillo");  
    yield return new WaitForSeconds(2);  
    Debug.Log("Verde");  
    yield return new WaitForSeconds(1);  
}
```

Corrutinas. Esperas con WaitForSeconds

- Podemos meter tantas esperas como queramos con la sintaxis:

yield return new WaitForSeconds

- s: Tiempo de espera en **segundos**.
- Durante estas “esperas”, el código principal se sigue ejecutando (método Update() y otros scripts)

Inicializar una corrutina.

- Para inicializar una corrutina utilizamos el método **StartCoroutine()**.
- **NO** deberíamos tener en ejecución **varias corrutinas** del mismo tipo a la vez. Por ejemplo:

```
void Update()  
{  
    StartCoroutine(Semaforo());  
}
```

- Crearíamos un semáforo nuevo **cada frame**, entrelazándose los diferentes estados de todos los semáforos que se están ejecutando a la vez.

Inicializar una corrutina en Update(). Solución

- Existen casos en los que queramos inicializar corrutinas en el Update().
- Para prevenir el solapamiento de esta, se utiliza una variable tipo **bool**.

```
void Update()
{
    if(Input.GetKeyDown(KeyCode.Space) && !enEjecucion)
    {
        StartCoroutine(RafagaTresDisparos());
    }
}

1 referencia
private IEnumerator RafagaTresDisparos()
{
    enEjecucion = true;
    //Instantiate primer disparo.
    yield return new WaitForSeconds(1f);
    //Instantiate segundo disparo.
    yield return new WaitForSeconds(1f);
    //Instantiate tercer disparo.
    enEjecucion = false;
}
```

Bucles infinitos en corrutinas.

- Debido a dichas esperas, el código “respira”, y podemos introducir comportamientos **infinitos** si lo deseamos.
- Para establecer bucles infinitos utilizaremos **while(true)**:

```
IEnumerator Semaforo()  
{  
    //Ahora nuestro semáforo funciona de forma infinita.  
    //Debido a los cortes, el programa no se colapsa.  
    while (true)  
    {  
        Debug.Log("Rojo");  
        yield return new WaitForSeconds(3);  
        Debug.Log("Amarillo");  
        yield return new WaitForSeconds(2);  
        Debug.Log("Verde");  
        yield return new WaitForSeconds(1);  
    }  
}
```


Otros bucles en corrutinas.

- Cualquier otro tipo de comportamiento cíclico con esperas/cortes es igual de válido. Por ejemplo:

```
IEnumerator Oleadas()
{
    //Habrá 10 niveles...
    for (int i = 0; i < 10; i++)
    {
        //De 3 oleadas cada nivel...
        for (int j = 0; j < 3; j++)
        {
            //De 10 enemigos cada oleada...
            for (int k = 0; k < 10; k++)
            {
                Instantiate(enemigo, spawner.transform.position, Quaternion.identity);
                yield return new WaitForSeconds(0.5f); //Espera entre cada enemigo...
            }
            yield return new WaitForSeconds(1); //Espera al terminar cada oleada...
        }
        yield return new WaitForSeconds(2); //Espera al terminar cada nivel...
    }
}
```


Corrutinas como "Update() finito"

- A veces es necesario ejecutar cierto código en el Update() pero **no** por siempre.
- Por ejemplo:

```
void Update()
{
    //Estaremos comprobando CONTINUAMENTE el if, aunque hayamos llegado al destino.
    if(transform.position != posicionDestino)
    {
        transform.position = Vector3.MoveTowards(transform.position, posicionDestino, velocidad * Time.deltaTime);
    }
}
```

Corrutinas como "Update() finito"

- En esos casos, podemos derivar la lógica a una corrutina.
- Los cortes de esta será una vez por frame, como el Update(), utilizando **yield return null**
- Cuando deje de cumplirse la condición, la corrutina terminará:

```
private void Start()
{
    StartCoroutine(MoverADestino());
}

1 referencia
private IEnumerator MoverADestino()
{
    //En cuanto lleguemos al destino, este while se rompe.
    while (transform.position != posicionDestino)
    {
        transform.position = Vector3.MoveTowards(transform.position, posicionDestino, velocidad * Time.deltaTime);
        yield return null;
    }
}
```

Parar corrutinas. StopAllCoroutines.

- Podemos para **todas** las corrutinas activas en un **script** con el método:

StopAllCoroutines();

- Inconvenientes:
 - No permite parar corrutinas de forma independiente.
- Solución:
 - StopCoroutine();

Parar corrutinas. StopCoroutine.

- Si queremos parar corrutinas de forma independiente, debemos guardar una referencia a **la llamada** que se pretende parar.
- Por ejemplo:

```
Coroutine llamada = StartCoroutine(Disparos());
```

- En el método **StopCoroutine()**, pasaremos por parámetro la **llamada** de la corrutina a parar:

```
StopCoroutine(llamada);
```

Método Start() como corrutina.

- ¿Sabías que...
 - El método Start() puede ser utilizado como corrutina?
- Simplemente, modifica su valor de salida por **IEnumerator**:

```
Mensaje de Unity | 0 referencias  
IEnumerator Start()  
{  
--  
}
```

Otros recursos.

- Countdown Timer in Unity:
- <https://www.youtube.com/watch?v=o0j7PdU88a4>
- Ways to Start & Stop Coroutines:
- https://www.youtube.com/watch?v=O_rya8qmQkw&t=1s&ab_channel=JasonWeimann
- C# Coroutines in Unity
- https://www.youtube.com/watch?v=5L9ksCs6MbE&t=41s&ab_channel=Unity